TASK 4 – Use Macros Safely

Safe macro examples:

```
#include <stdio.h>

#define RUPEES_TO_PAISE(r) ((long long)(r) * 100LL)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

int main(void)
{
    int a = 3;
    int b = 7;
    printf("Square: %d\n", SQUARE(a + 1));
    printf("Maximum: %d\n", MAX(a, b));
    printf("Rupees to paise: %lld\n",
RUPEES_TO_PAISE(50));
    return 0;
}
```

Expected output:

```
Square: 16
Maximum: 7
Rupees to paise: 5000
```

Common macro pitfalls:

- Without parentheses, #define SQUARE(x) x*x makes SQUARE(a+1) expand to a+1*a+1. With a = 3, this evaluates to 3 + 3 + 1 = 7 rather than 16. Parentheses around the parameter and the whole expression fix precedence.
- MAX(a, b) can evaluate an argument more than once. For example, MAX(i++, j) may increment i during the comparison and again if the first argument is selected. Avoid side effects in macro arguments.
- Macros are expanded by the preprocessor. Run gcc -E file.c -o file.i and inspect file.i to see the expanded source.
- A function is usually better when arguments may have side effects, type checking matters, or the logic is more than a tiny expression.

Inline function alternative and complexity:

```
static inline int square_int(int x)
{
    return x * x;
}

static inline int max_int(int a, int b)
{
    return (a > b) ? a : b;
}
```

Both a macro expansion and a function call perform constant-time work for these operations: O(1). A compiler may inline an inline function, but it is not required to do so. A macro's expanded code may be repeated at every use, increasing source and potentially machine-code size when used many times. An inline function has one definition at source level and gives type checking; optimisation may remove call overhead. For this assignment, use macros for simple compile-time constants and prefer functions for reusable calculations.

4.4 Preprocessor evidence:

```
gcc -E task4.c -o task4.i
# Open task4.i and locate the expanded SQUARE / MAX expressions
gcc -Wall -Wextra -std=c11 task4.c -o task4
./task4
```